

DevFlow AI

20 herramientas para developers

Trabajo Final de Master — Desarrollo con IA (BIG School)

Alberto Guinda Sevilla · Marzo 2026



El Problema

Los developers usan 10+ herramientas fragmentadas para tareas cotidianas

Desarrollo con IA

- Prompts sin evaluar calidad
- Costes de API opacos
- Tokenizacion invisible
- Context windows mal gestionados

Tareas diarias

- Formatear JSON, Base64, UUID...
- Ordenar clases Tailwind
- Construir expresiones cron
- Generar commits convencionales

Resultado: Friccion, costes ocultos, perdida de productividad

La Solucion

Una plataforma unificada, gratuita y open-source



Local-first

Todo en el navegador. Sin login. Sin API keys.



IA opcional

4 proveedores con fallback.
Pollinations siempre gratis.



Enterprise-grade

CSP, HSTS, SAST, CodeQL,
rate limiting, Sentry.

20 herramientas · 1780 tests · Lighthouse 100/100/100/100

Las 20 Herramientas

****JSON Formatter**** — Format, diff, tree view

****Variable Name Wizard**** — 8 convenciones

****Regex Humanizer**** — Explicacion + railroad

****Code Review**** — Smells, complejidad

****Cost Calculator**** — 10+ modelos IA

****Base64**** — URL-safe, data URL

****UUID Generator**** — v1-v5, v7, bulk

****DTO-Matic**** — JSON → TS + Zod

****Git Commit**** — Convencional + emojis

****Cron Builder**** — Visual + calendario

****Tailwind Sorter**** — Ordena + diff

****Prompt Analyzer**** — Score + inyeccion

****Token Visualizer**** — BPE + costes

****Context Manager**** — Chunks + export

****HTTP Status**** — 61 codigos + guias

****Hash Generator**** — SHA/MD5/HMAC

****JWT Decoder**** — Decode + validar

****Color Converter**** — HEX/RGB/HSL/OKLCH

****Diff Comparer**** — LCS line diff

****Password Generator**** — Criptografico

Cada herramienta sigue el patron 5-capas: types → lib/application → hooks → page → tests

Arquitectura: Clean Architecture

Patron 5-capas por herramienta

```
types/<tool>.ts           → Interfaces
lib/application/<tool>.ts → Logica pura
hooks/use-<tool>.ts       → Estado + localStorage
app/ ... /page.tsx        → UI
tests/ ... /<tool>.test.ts → Tests
```

Flujo: Presentation → Application → Domain

0 violaciones de capas en las 20 herramientas

AI Backend (opcional)

```
hooks/use-ai-<feature>.ts → SWRMutation
app/api/ai/<endpoint>/    → Route Handler
```

Provider chain:

1. BYOK (tu API key)
2. Gemini 2.0 Flash
3. Groq
4. OpenRouter
5. Pollinations (gratis)

Rate limiter: 10 RPM / 50 RPM BYOK

Stack Tecnológico

Capa	Tecnologia
Framework	Next.js 16 (App Router)
UI	React 19 (Compiler)
Lenguaje	TypeScript 5 (max strict)
Estilos	Tailwind CSS v4
Componentes	HeroUI v3 beta
Animaciones	GSAP + Framer Motion

Capa	Tecnologia
Testing	Vitest + Playwright
a11y	axe-core WCAG AAA
CI/CD	GitHub Actions (10 jobs)
Error tracking	Sentry
Hosting	Vercel Edge
State	Zustand + Context

21 deps produccion · TypeScript maximum strict (15+ flags) · Zero `any`

Server Components & Performance

Homepage (Server Component)

page.tsx (async RSC)

```
├── Hero      → Server (0 JS)
├── Stats     → Server + GSAP island
│   └── GitHub → Server fetch (ISR 1h)
├── Features  → Client (GSAP stagger)
├── Why DevFlow → Server (0 JS)
└── Footer    → Server (0 JS)
```

Resultado: Hero y Footer = 0 bytes JS

Lighthouse Desktop

Metrica	Score
Performance	100
Accessibility	100
Best Practices	100
SEO	100

React Compiler + View Transitions API + ISR +
optimizePackageImports

Testing: Estrategia 100/80/0

Piramide estrategica

Tier	Target	Path
CORE	80-100%	lib/application/*.ts
IMPORTANT	80%	components/shared/
INFRA	0%	types/ , config/

Per-file enforcement: cada archivo CORE debe cumplir individualmente

CI bloquea merge si cualquier archivo baja del umbral

Metricas actuales

```
Tests:    1780 passing (53 files)
E2E:      25 Playwright specs
a11y:     20 paginas (axe-core WCAG AA)
Duration: ~72s
```

Top herramientas:

- Regex Humanizer: 150 tests
- UUID Generator: 112 tests
- Cron Builder: 111 tests
- DTO-Matic: 108 tests
- Tool Recommendations: 101 tests

Seguridad

HTTP Headers

- CSP estricto (sin `unsafe-eval`)
- HSTS 2 anos + preload
- X-Frame-Options DENY
- Permissions-Policy restrictivo
- Prototype pollution protegido

Application

- Rate limiting IP-based
- Zod validation en endpoints
- Anti-injection en system prompts
- Zero `eval()` / `innerHTML`

CI/CD Security (10 jobs)

- `npm audit --audit-level=high`
- lockfile-lint validacion registro
- CycloneDX SBOM en cada build
- CodeQL SAST JS/TS
- Semgrep OWASP Top 10
- eslint-plugin-security
- StepSecurity harden-runner
- SHA-pinned todas las Actions
- Dependency review en PRs
- Renovate auto-updates

Internacionalizacion

Sistema custom (sin i18next)

- 8 idiomas: EN, ES, FR, PT, DE, IT, ZH, JA
- ~1590 claves por idioma (x8 archivos)
- Hook `useTranslation()` con interpolacion
- Deteccion automatica del navegador
(`navigator.language`)
- Cambio instantaneo via Zustand + Dropdown con banderas

Cobertura

- 20 tool pages completamente traducidas
- Landing page, settings, docs, history
- ARIA labels, placeholders, toasts
- Error messages, empty states

Ejemplo

```
// locales/en.json (x8 archivos)
{ "costCalc.cachedPrices": "Using cached prices" }

// locales/ja.json
{ "costCalc.cachedPrices": "キャッシュ価格を使用中" }

// En el componente
const { t } = useTranslation();
<Chip>{t("costCalc.cachedPrices")}</Chip>
```

0 strings hardcodeadas en produccion

Features Clave

Command Palette (`Cmd+K`)

- Búsqueda fuzzy de 20 tools + 5 acciones
- Navegación por teclado completa
- Zero dependencias extra

Smart Suggestions

- 10 detectores de tipo de dato
- 15 reglas de recomendación
- Data passing entre herramientas

PWA

- Manifest + Service Worker
- Instalable como app standalone
- Offline-capable

Export/Import Settings

- Backup de todas las preferencias
- Validación Zod al importar
- Filtro `devflow-*` keys

MagicInput

- Detección automática de tipo
- Redirige a la herramienta correcta
- JSON, Cron, Regex, Base64...

Share via URL

- Compresión gzip + base64url
- Web Share API + clipboard fallback
- Las 20 herramientas

Keyboard Shortcuts

- `Ctrl+Enter` ejecutar, `Ctrl+Shift+C` copiar

Resultados

20

Herramientas

1780

Tests passing

40

Rutas

~1590

Claves i18n (x8)

10

Jobs CI/CD

100

Lighthouse (x4)

4 proveedores IA · WCAG AAA · PWA instalable · Zero any

Conclusiones

Logros principales

1. Producto funcional — 20 tools en produccion
2. Clean Architecture — 5-capas sin excepciones
3. Performance maxima — Lighthouse 100x4
4. Testing robusto — 1780 + 25 E2E + a11y
5. Seguridad enterprise — SAST, CSP, rate limiting
6. UX avanzada — PWA, Cmd+K, Share URL, shortcuts
7. IA opcional — 4 proveedores, BYOK, fallback
8. i18n 8 idiomas — ~1590 claves x8, deteccion automatica
9. SEO/GEO — JSON-LD, llms.txt, dominio propio

Aprendizajes clave

- RSC reduce drasticamente el JS al cliente
- Clean Architecture vale la pena para testing
- TypeScript strict previene categorias de bugs
- 100/80/0 es mas sostenible que 100% global
- Claude Code como pair programmer acelera significativamente
- 4 auditorias (79+ fixes) llevan a 0 vulnerabilidades

Trabajo futuro

- Cloud sync (Supabase)
- Team collaboration
- Browser extension
- Mobile app (React Native)

Demo en vivo

devflowai.dev

Prompt Analyzer

Code Review

Cost Calculator

Token Visualizer

JSON Formatter

Context Manager

Repo: github.com/albertoguinda/devflow-ai

Preguntas

Recurso	Enlace
Demo	devflowai.dev
Repositorio	github.com/albertoguinda/devflow-ai
TFM Documento	docs/TFM.md